TCSS 321 – Discrete Structures

Insertion Sort

Insertion Sort

Sort a List

Input: A sequence of integers $A = (a_1, a_2, \dots, a_n)$.

Output: A' a permutation of A such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

- Can we state the insertion sort algorithm as a recursive algorithm?

InsertionSort$(A = (a_1, a_2, \dots, a_n))$

    If $n \leq 1$ return A.

    Let $A' =$ InsertionSort$(A = (a_1, a_2, \dots, a_{n-1}))$.

    A'.add(a_n).

    Let $j = n$.

    While $a_j < a_{j-1}$ and $j > 1$ do:

        Swap$(j,\ j-1)$.

        Let $j = j - 1$.

    Endfor

Return A'.

Insertion Sort

```
InsertionSort(A = (a_1, a_2, ..., a_n))
        If n ≤ 1 return A.
        Let A' = InsertionSort(A = (a_1, a_2, ..., a_{n-1})).
        A'.add(a_n).
        Let j = n.
        While j > 1 and a_j < a_{j-1} do:
                Swap(j, j − 1).
                Let j = j − 1.
        Endfor
Return A'.
```

- Can we express the number of operations carried out by this algorithm as a recurrence relation?

$$T(n) = \begin{cases} 3 & \text{if } n \leq 1 \\ T(n-1) + 11 + \sum_{j=2}^{n} 8 & \text{if } n > 1 \end{cases} = \begin{cases} 3 & \text{if } n \leq 1 \\ T(n-1) + 8n + 3 & \text{if } n > 1 \end{cases}$$

Insertion Sort

$$T(n) = \begin{cases} 3 & \text{if } n \leq 1 \\ T(n-1) + 8n + 3 & \text{if } n > 1 \end{cases}$$

- Let's investigate some of the values of $T(n)$:

$$T(1) = 3$$
$$T(2) = T(1) + 16 + 3 = 22$$
$$T(3) = T(2) + 24 + 3 = 49$$
$$T(4) = T(3) + 32 + 3 = 84$$
$$T(5) = T(4) + 40 + 3 = 127$$

- Can we see a pattern?
  - As the recurrence relation gets more complex it becomes harder to guess the pattern.
  - In Design and Analysis of Algorithms we will investigate two techniques to find the pattern: recursion trees, algebraic substitution.

Insertion Sort

$$T(n) = \begin{cases} 3 & \text{if } n \leq 1 \\ T(n-1) + 8n + 3 & \text{if } n > 1 \end{cases}$$

- Let's see how we can re-express $T(n)$ with less recursion:

$$T(n) = T(n-1) + 8n + 3$$
$$= T(n-2) + 8(n-1) + 3 + 8n + 3$$
$$= T(n-3) + 8(n-2) + 3 + 8(n-1) + 3 + 8n + 3$$
$$\dots$$
$$= T(n-i) + \sum_{j=0}^{i-1} [8(n-j) + 3]$$
$$= T(1) + \sum_{j=0}^{n-2} [8(n-j) + 3]$$
$$= 3 + \sum_{j=0}^{n-2} [8(n-j) + 3]$$
$$= 3 + \sum_{j=0}^{n-2} [8j + 3]$$
$$= 3 + \sum_{j=0}^{n-2} 8n - \sum_{j=0}^{n-2} 8j + \sum_{j=0}^{n-2} 3$$
$$= 3 + 8n \sum_{j=0}^{n-2} 1 - 8 \sum_{j=0}^{n-2} j + 3 \sum_{j=0}^{n-2} 1$$
$$= 3 + 8n(n-1) - \frac{8(n-1)(n-2)}{2} + 3(n-1)$$
$$= 3 + 8n^2 - 8n - 4n^2 + 12n - 8 + 3n - 3$$
$$= 4n^2 + 7n - 8$$

Upper Bound

$$T(n) = \begin{cases} 3 & \text{if } n \leq 1 \\ T(n-1) + 8n + 3 & \text{if } n > 1 \end{cases}$$

Theorem: $T(n) \leq bn^2$ for all $n > n_0$.

Proof: (by induction on n)

Base case: $(n = 1)$

$$T(1) = 3 \leq b \cdot 1^2 = b$$

This is only true if $b \geq 3$.

Inductive Hypothesis: Assume that $T(k) \leq bk^2$ for all $k < n$.

Inductive case: $(n > 1)$

$$\begin{aligned} T(n) &= T(n-1) + 8n + 3 \\ &\leq b(n-1)^2 + 8n + 3 \\ &= bn^2 - 2bn + b + 8n + 3 \\ &\leq bn^2 - 2bn + bn + 8n + 3n \\ &\leq bn^2 - bn + 11n \\ &\leq bn^2 \end{aligned}$$

The last step is true only if $b \geq 11$. So we select $b = 11$ and $n_0 = 1$.

By induction $T(n) \leq bn^2$ for all $n > n_0$. ■

Lower Bound

$$T(n) = \begin{cases} 3 & \text{if } n \leq 1 \\ T(n-1) + 8n + 3 & \text{if } n > 1 \end{cases}$$

Theorem: $T(n) \geq an^2$ for all $n > n_0$.

Proof: (by induction on n)

Base case: $(n = 1)$

$$T(1) = 3 \geq a \cdot 1^2 = a$$

This is only true if $a \leq 3$.

Inductive Hypothesis: Assume that $T(k) \geq ak^2$ for all $k < n$.

Inductive case: $(n > 1)$

$$\begin{aligned} T(n) &= T(n-1) + 8n + 3 \\ &\geq a(n-1)^2 + 8n + 3 \\ &= an^2 - 2an + a + 8n + 3 \\ &\geq an^2 - 2an + 8n \\ &\geq an^2 \end{aligned}$$

The last step is true only if $a \leq 4$. So we select $a = 3$ and $n_0 = 1$.

By induction $T(n) \geq an^2$ for all $n > n_0$. ■

TCSS 321 – Discrete Structures

Merge Sort

Merge Sort

Sort a List

Input: A sequence of integers $A = (a_1, a_2, \dots, a_n)$.

Output: A' a permutation of A such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

- Can we state the Merge sort algorithm as a recursive algorithm?

MergeSort$(A = (a_1, a_2, \dots, a_n))$

If $n \leq 1$ return A.

Let $B =$ MergeSort$\left(A = \left(a_1, a_2, \dots, a_{\lfloor \frac{n}{2} \rfloor}\right)\right)$.

Let $C =$ MergeSort$\left(A = \left(a_{\lfloor \frac{n}{2} \rfloor + 1}, a_{\lfloor \frac{n}{2} \rfloor + 2}, \dots, a_n\right)\right)$.

Return Merge(B, C).

- Here we will assume Merge is a subroutine that merges two sorted lists into one sorted list using a number of operations in $\Theta(n + m)$ where $n = |A|$ and $m = |B|$.

Merge Sort

```
MergeSort(A = (a_1, a_2, ..., a_n))
        If n ≤ 1 return A.
        Let B = MergeSort(A = (a_1, a_2, ..., a_⌊n/2⌋)).
        Let C = MergeSort(A = (a_⌊n/2⌋+1, a_2, ..., a_n)).
Return Merge(B, C).
```

- Here we will assume Merge is a subroutine that merges two sorted lists into one sorted list using a number of operations in $\Theta(n+m)$ where $n = |A|$ and $m = |B|$.
- Can we express the number of operations of the Merge sort algorithm as a recurrence relation?

$$T(n) = \begin{cases} 3 & \text{if } n \leq 1 \\ T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + d\left(\left\lfloor \frac{n}{2} \right\rfloor + \left\lceil \frac{n}{2} \right\rceil\right) + 8 & \text{if } n > 1 \end{cases}$$

- To simplify this recurrence we will assume n is a power of 2 which will allow us to eliminate the floor and ceiling.

$$T(n) = \begin{cases} 3 & \text{if } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + dn & \text{if } n > 1 \end{cases}$$

Upper Bound

$$T(n) = \begin{cases} 3 & \text{if } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + dn & \text{if } n > 1 \end{cases}$$

Theorem: $T(n) \leq bn \log_2 n + a$ for all $n \geq n_0$.

Proof: (by induction on n)

Base case: $(n = 1)$

$$T(1) = 3 \leq b1 \log_2 1 + a = a$$

This step is only true when $a \geq 3$.

Inductive Hypothesis: Assume $T(k) \leq bk \log_2 k + a$ for all $k < n$.

Inductive case: $(n > 1)$

$$T(n) = 2T\left(\frac{n}{2}\right) + dn$$

$$\leq 2\left(b\left(\frac{n}{2}\right)\log_2\left(\frac{n}{2}\right) + a\right) + dn$$

$$\leq bn \log_2\left(\frac{n}{2}\right) + 2a + dn$$

$$\leq bn \log_2(n) - bn \log_2(2) + dn + 2a$$

$$= bn \log_2 n + a - bn + dn + a$$

$$\leq bn \log_2 n + a$$

Note that the last line is only true if

$$-bn + dn + a \leq 0$$

or if

$$dn + a \leq bn$$

which we note is true when

$$d + a \leq b$$

so we select $a = 3$ and $b = d + 3$ and $n_0 = 1$.

Then by induction $T(n) \leq bn \log_2 n + a$ for all $n \geq n_0$. ■

Lower Bound

$$T(n) = \begin{cases} 3 & \text{if } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + dn & \text{if } n > 1 \end{cases}$$

Theorem: $T(n) \geq an \log_2 n$ for all $n \geq n_0$.

Proof: (by induction on n)

Base case: $(n = 1)$

$$T(1) = 3 \geq a1 \log_2 1 = 0$$

This step is true for any a.

Inductive Hypothesis: Assume $T(k) \geq ak \log_2 k$ for all $k < n$.

Inductive case: $(n > 1)$

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + dn \\ &\geq 2\left(\frac{an}{2} \log_2 \frac{n}{2}\right) + dn \\ &\geq an \log_2(n) - an + dn \\ &\geq an \log_2 n \end{aligned}$$

The last step is true only if $a \leq d$. So we will select $a = d$ and $n_0 = 1$.

Then by induction $T(n) \leq bn \log_2 n$ for all $n \geq n_0$. ■